A Smaller Validated Checker for the Concise iPRA Axiomatization

Research note prepared for Vladimir Reshetnikov

11 November 2026

Abstract

The verified improvement in this note is a smaller complete implementation of the existing ConciseIPRA presentation. The fully validated Eq/Q/D checker has 870 normalized Python bytes, 383 significant tokens, and 442 AST nodes, against 897 bytes, 401 tokens, and 468 nodes in the newest matching repository archive. Its maximum-compression gzip size is 507 bytes, compared with 505. The Eq/D public-language variant has 849 bytes, compared with 878. The reduction replaces the recursive raw-type mode of the validator by an exact characterization of integer/list JSON trees using their Python representation. The arithmetic proof kernel is unchanged.

This is a modest reduction under the complete-proof-checker criterion. It is not a new smaller set of arithmetic axioms, a mathematical breakthrough, an absolute minimum, or an improvement over the separate 521-byte guarded checker, which has a different validation contract. The note states the complete inherited axiomatization, both interpretations and their composites, the new validator-preservation proof, measured comparisons, and reproducible checks. Attributed proof appendices make the mathematical dependencies available inside the archive.

Result at a glance

Full Eq/Q/D presentation	Bytes	Tokens	AST	Gzip-9
Pinned incoming control	897	401	468	505
This presentation	**870**	**383**	**442**	507
Reduction	27	18	26	−2

All four columns count a complete standalone program, including parsing, validation, proof checking, and the acceptance output. Gzip is informational. The negative last entry means that compressed size increases by two bytes.

Contents

1 The question and the comparison boundary **4**

2 The mathematical theory implemented by the checker **4**

2.1 Finite syntax and all primitive rules 5

2.2 Where the ordinary logical operations come from 6

2.3 The ordinary first-order realization 7

2.4 The arithmetic bridge to standard iPRA 7

3 A smaller validation boundary **9**

3.1 Recognizing integer/list trees without a recursive type mode 9

3.2 Specializing the scoped goal validator 9

4 The complete standalone implementation **11**

4.1 Why a tempting additional reduction was rejected 11

5 Measurements and validation **12**

5.1 Measurement convention 12

5.2 What the executable checks establish 12

6 Further research questions **13**

Artifact guide and attribution **13**

A Complementary introductions with one local assumption **14**

A.1 Complete native syntax and rule inventory 14

A.2 Exact equivalence and the unused-witness construction 15

A.3 Induction and numerical cases separate cleanly 16

A.4 The entire ordinary logical interface 17

A.5 Whole-domain realizations and both formula composites 18

A.6 What the theorem does and does not claim 19

B Arithmetic strength and both ordinary interpretation composites **19**

B.1 The arithmetic dependency order 20

C Internal scalar predecessor bootstrap **21**

C.1 Elementary terms and two finite cycles 21

C.2 A ramp with a global scaling identity 22

C.3 A comparator from scalar iteration 22

C.4 Saturated iteration gives predecessor 23

C.5 Dependency boundary 24

D From literal-equation induction to full intuitionistic PRA **24**
D.1 The exact hypotheses . . . 25
D.2 Constructor separation and two elementary tests . . . 25
D.3 Four elementary uses of equation induction . . . 26
D.4 An equation invariant that reflects actual equality . . . 27
D.5 Compilation of all quantifier-free formulas . . . 28
D.6 Recovery of induction and the full theory . . . 29

E Pairing, primitive recursion, and internal numerical composites **30**
E.1 Pairing from a bounded square-root search . . . 30
E.2 Compiling each primitive recursion . . . 31
E.3 Why both numerical round trips are internal . . . 31

F Removing literal zero from the source of transport **31**
F.1 Structural operations, cases, and equation induction . . . 32
F.2 The equality operations available immediately . . . 33
F.3 A singleton-one test without predecessor . . . 33
F.4 Turning a zero-source equality around . . . 34
F.5 Recovering equality algebra and formula transport . . . 35
F.6 Compatibility with restricted computation certificates . . . 36
F.7 Scope of the result . . . 36

G A uniform arity encoding and its checking theorem **37**
G.1 The complete sorted interpretation of arrays . . . 37
G.2 One depth calculation replaces the lookup table . . . 38
G.3 Opening and scope . . . 38
G.4 A computation hypothesis only needs a zero instance . . . 39
G.5 Audit of the logical continuations . . . 40
G.6 Canonical completeness, including zero witnesses . . . 41
G.7 JSON and finite-runtime qualifications . . . 42

1 The question and the comparison boundary

The requested objective is a simpler axiomatization of full first-order intuitionistic primitive recursive arithmetic, with equiconsistency and bi-interpretability, while allowing a different language or numerical domain. The recurring operational simplicity criterion is the size of a complete, straightforwardly inspectable proof checker. We distinguish that criterion from the number of primitive mathematical rules and from the size of a proof certificate.

The comparison is pinned to ProveIt commit `42a48d2f04ad72036e8d7e365ea91ea08c76a9e3`, specifically `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The current README describes 38 research packages and a Lean development. In addition, the incoming directory contains the newer `ipra_one_sided_transport_2026-11-11.zip` and `ipra_validated_core_2026-11-11.zip`. Both were inspected before selecting the control. The latter supplies the 897-byte full native and 878-byte Eq/D checkers used here. Exact repository identifiers, copied source hashes, and the incoming Git blob identities are recorded in `reference/provenance.json`.

The full native public language has equality, Q, and D. The typed input contract also requires every node of the whole certificate, including an unused prefix, to be an exact Python integer or list after JSON parsing. In particular, Boolean values, floating-point values, strings, dictionaries, and null are rejected. The advertised goal must be a canonical closed formula. A shorter program obtained by omitting one of these checks would not be a matching improvement.

The separate compact guarded envelope is 256 significant tokens, 278 AST nodes, and 521 normalized bytes, attained by three different programs. Those programs assume a canonical guarded public goal. None of the present claims compares 870 against 521 as though the input contracts were equal. The current seven-rule mathematical variants also remain available; this note does not assert that its unchanged mathematical basis has fewer rules.

Status of the requested mathematical simplification. No further reduction of the arithmetic or logical axiom basis was established in this investigation. Removing the initializer, eliminating primitive successor, replacing case introduction by uniform progress, and narrowing transport have already been extensively investigated in the pinned sources. The positive result below is a proved simplification of a complete validated presentation of the same theory. This distinction is part of the result, not an implicit claim of a new foundation.

2 The mathematical theory implemented by the checker

The reduction in this report concerns the complete validated proof checker. It does not remove a mathematical axiom, weaken a side condition, or introduce a new arithmetic principle. For clarity, this section states the existing theory in full and identifies the inherited proof of its relationship with iPRA. The source is the ConciseIPRA snapshot at commit `42a48d2f04ad72036e8d7e365ea91ea08c76a9e3`. The attributed foundational proofs are reproduced in the appendices.

The presentation below is the eight-rule calculus called `IPrf` in the repository. The existing `NPrf` kernel also has eight rules, with weaker equational transport and direct equation-progress introduction; `iprf_iff_nprf` proves that their consequence relations coincide. The repository additionally has equivalent seven-rule presentations. None of those mathematical rule counts is improved here.

2.1 Finite syntax and all primitive rules

There is one numerical sort. Its terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in the update body h; it is not bound in n or a. Every other parameter of h stays fixed during the iteration. A body is a fixed finite expression, not a quantified code for a function. In particular, this syntax supplies no uniform evaluator for arbitrary primitive-recursive descriptions. All substitutions below avoid capture, and bound variables may be renamed.

The native formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The displayed variables are bound in the matrix of their respective binder; x and y in D are distinct. A context Γ is a finite reusable list of formulas. Write $1 := S0$ and designate $0 = 1$ as native contradiction. Neither ordinary implication nor ordinary falsity is primitive syntax here.

Here is the entire primitive inventory. The context in each conclusion is the same retained context as in its premises.

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma) \qquad \text{Hyp},$$

$$\frac{}{\Gamma \vdash I(0, a; u.h) = a} \qquad I_0,$$

$$\frac{}{\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]} \qquad I_S,$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \qquad \text{Tr},$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)} \qquad \text{QE},$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A} \qquad \text{DE},$$

$$\frac{\Gamma, B(t, 0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)} \qquad \text{DI}_{\text{case}},$$

$$\frac{\Gamma, E(t, z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E(x, y)} \qquad \text{DI}_{\text{progress}}.$$

In the last rule E must be a *literal equation*: its outermost constructor is equality. An equation occurring inside Q or D does not meet that condition. In Tr, $F(v)$ is any native formula template with one designated free numerical variable; that variable may occur any finite number of times, including zero. It is renamed away from the substituted terms and the retained context. The term t in QE is arbitrary.

In DE, the eigenvariable b is fresh for Γ, the conclusion A, and the passive parameters of B. In either DI rule, z is fresh for Γ, the witness t, and the passive parameters of the matrix. The displayed local assumption is discharged. These freshness conditions prevent a witness from depending on an eigenvariable introduced later.

There are no other primitive equality axioms, successor axioms, induction schemas, logical connectives, or logical rules. Weakening, contraction, exchange, fresh renaming, substitution, and assumption cut are derived operations. For example, if $r = I(0, t; u.0)$, then I_0 proves $r = t$; using that equation as both premises of Tr with $F(v) \equiv (v = t)$ proves $t = t$. Symmetry, transitivity, and congruence then follow by the usual equality templates. The full derivations are in Section A.

2.2 Where the ordinary logical operations come from

The scalar selector

$$C(v,a,b) := I(v,a;u.b), \qquad p(y) := I(y,0;u.C(u,1,0))$$

has $C(0,a,b) = a$ and $C(Sv,a,b) = b$; the update of C ignores u. The term p is the parity iterator, and only its fixed instances $p(1) = 1$ and $p(2) = 0$ are needed for the conjunction macro below.

A finite syntactic gate $G_v A$ and selector $M_v(A,B)$ satisfy

$$\vdash G_0 A, \qquad G_{Sv} A \dashv\vdash A, \qquad M_0(A,B) \dashv\vdash A, \qquad M_{Sv}(A,B) \dashv\vdash B.$$

Here interderivability holds under arbitrary additional retained assumptions. At equations their definitions are

$$G_v(s = t) := (C(v,0,s) = C(v,0,t)),$$
$$M_v(s = t, r = q) := (C(v,s,r) = C(v,t,q)).$$

For a completely finite extension to all formulas, first replace each $Q_y B$ by the interderivable $D_{r,y} B$, with r unused and fresh. The gate commutes with D. To align selectors of unequal binder depth, pad an equation E by $D_{r,y} G_y E$; then recurse under the aligned D binders. Section A.4 gives the exact terminating recursion, the padding equivalence, and its proof. This is external recursion on fixed formula syntax, not an internal truth predicate.

Using those operations, one may take the ordinary logical macros to be

$$\begin{aligned}
\bot_* &:= (0 = 1),\\
A \to_* B &:= Q_y M_y(A,B),\\
A \wedge_* B &:= Q_y G_y M_{p(y)}(A,B),\\
\exists_* x\, A(x) &:= D_{x,y} G_y A(x),\\
A \vee_* B &:= \exists_* v\, M_v(A,B),\\
\forall_* x\, A(x) &:= A(0) \wedge_* Q_x A(x).
\end{aligned}$$

All new bound variables are fresh; before taking a selector or gate, use the equivalent D-only version just described when necessary. The equivalent D-only inventory in Section A.4 also avoids treating Q as a public formula constructor.

These macros have every intuitionistic natural-deduction rule. Implication introduction uses the zero selector as its local antecedent and supplies the successor selector as the consequent; elimination specializes a closure at 1. Existential introduction uses its supplied witness, while DE opens a fresh witness for existential elimination; the zero gate supplies the required base. Conjunction projections use closure instances 1 and 2. Disjunction uses witnesses 0 and 1, followed by the derived numerical case rule for elimination. A fresh-parameter proof gives universal introduction, and its zero instance together with its closure gives arbitrary specialization. Under $0 = 1$, the selector equations give $s = C(0,s,t) = C(1,s,t) = t$; external formula induction then proves explosion. No excluded middle for quantified formulas is introduced.

The resulting numerical case principle and literal-equation induction are

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)},$$

with the usual fresh z and with E a literal equation. In the second construction, progress DI with an unused witness supplies a closure, and QE applies it to the given base. The first construction uses case DI and QE. One must not silently apply the first rule with z free in a retained assumption. The inherited proofs explicitly discharge that assumption using the implication macro or use coverage with equality hypotheses instead.

2.3 The ordinary first-order realization

To state bi-interpretability in ordinary first-order terms, elaborate each fixed iterator body $h(u, \vec{p})$ to a staged function symbol $i_h(n, a, \vec{p})$ satisfying

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finitely described bodies are introduced at each stage. This elaboration changes notation, not the numerical domain.

Let T_{eq} be intuitionistic first-order logic with actual equality in this signature, the displayed computation schemas, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

It also has induction for every literal equation $E(n, \vec{p})$:

$$\big(E(0, \vec{p}) \wedge \forall n\,(E(n, \vec{p}) \to E(Sn, \vec{p}))\big) \to \forall n\, E(n, \vec{p}),$$

universally closed over the passive parameters. Its contextual induction rule is the equivalent displayed rule above. The separation sentence is essential: it connects native contradiction to genuine ordinary falsity. The equation-only part by itself must not be substituted for this theory.

The ordinary reading ρ fixes terms and equality and is given by

$$\rho(Q_y B) = \rho(B(0)) \to \forall y\, \rho(B(y)),$$

$$\rho(D_{x,y} B) = \exists x\big(\rho(B(x, 0)) \to \forall y\, \rho(B(x, y))\big).$$

Case DI is sound by coverage. Progress DI is sound by equation induction under a temporarily retained base equation. The other rules have their ordinary equality or logical meanings. Conversely, the logical macros, derived numerical cases, and equation induction recover the ordinary realization. The native translation λ of ordinary formulas fixes equations and replaces the six ordinary logical operations by the macros above. Sections A and B prove preservation of sequents and both formula composites, not merely agreement in the standard natural numbers.

2.4 The arithmetic bridge to standard iPRA

Here standard iPRA means intuitionistic first-order arithmetic with one function symbol for each fixed primitive-recursive description: zero, successor, projections, composition, and primitive recursion. It has their defining equations, $Sx \neq 0$, successor injectivity, and induction for every quantifier-free formula, with parameters. The usual intuitionistic logical rules retain assumptions and enforce fresh variables for universal introduction and existential elimination.

The inherited arithmetic bridge has a necessary dependency order. First, a scalar term P is constructed and proved to satisfy $P0 = 0$ and $P(Sn) = n$, using no pre-existing pairing or equality detector (Section C). Next, truncated subtraction and a Boolean term reflecting actual numerical equality are constructed. Every fixed quantifier-free formula is then equivalent to its Boolean characteristic equation, so equation induction yields quantifier-free induction (Section D). Only after that step are pairing and its two projections used to compile general primitive recursion (Section E).

For example, let π, L, H satisfy $L(\pi(a, b)) = a$ and $H(\pi(a, b)) = b$. A target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is compiled, after its earlier descriptions g, h have been compiled, by

$$\begin{aligned} W(u,\vec{p}) &:= \pi(SL(u), h^*(L(u), H(u), \vec{p})),\\ V(n,\vec{p}) &:= I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})),\\ f^*(n,\vec{p}) &:= H(V(n,\vec{p})). \end{aligned}$$

Equation induction proves $L(V(n,\vec{p})) = n$, after which the projection equations prove both target defining equations. The compiler is external recursion on each finite description. Pairing stores a finite state; the interpretation domain is not restricted to the image of π.

In the other direction, iPRA expresses every scalar iterator by ordinary primitive recursion whose update ignores the recursion counter. It proves coverage, separation, and equation induction. Thus each theory interprets the other, and the following stronger statement supplies the required round trips.

Theorem 2.1 (Inherited whole-domain bi-interpretation)**.** *There are effective translations Φ from native formulas to iPRA and Ψ from iPRA to native formulas such that, for all finite contexts,*

$$\Gamma \vdash A \quad\Longleftrightarrow\quad \Phi\Gamma \vdash_{\mathrm{iPRA}} \Phi A,$$

$$\Delta \vdash_{\mathrm{iPRA}} \varphi \quad\Longleftrightarrow\quad \Psi\Delta \vdash \Psi\varphi.$$

Every native A is interderivable with $\Psi\Phi A$, and iPRA proves $\varphi \leftrightarrow \Phi\Psi\varphi$ for every ordinary formula φ. Both interpretations use the entire numerical domain and actual equality. In each direction the internally definable numerical return isomorphism has graph $x = y$. In particular,

$$\varnothing \vdash 0 = S0 \quad\Longleftrightarrow\quad \varnothing \vdash_{\mathrm{iPRA}} \bot.$$

Proof. Compose the native/ordinary translations ρ, λ with the two arithmetic compilations just described. For a fixed iterator, equation induction proves uniqueness from its seed and update equation. The same argument proves uniqueness of a fixed ordinary primitive recursion. External induction on a finite description therefore shows that its double translation equals its original value at every internal argument. This is an internal numerical equality, valid at nonstandard arguments as well as standard numerals.

The identity numerical graph is consequently total, bijective, and preserves every interpreted operation. The logical macro equivalences and binder composites then give both formula round trips by external structural induction. In particular, $D_{x,y}B$ is interderivable with the native existential of the native implication from $B(x,0)$ to the native universal of $B(x,y)$; the analogous closure statement holds for Q. The explicit ordinary separation sentence and native explosion preserve contradiction. Finite proof translations establish equiconsistency. The detailed constructions are in Sections A, B, and E. □

The pinned Lean development records the ordinary arithmetic bridge as `biInterp`, `teq_roundTrip`, and `pra_roundTrip`; the native bridge is recorded by `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff` in `NativeBiInterpretation.lean` and its dependencies. The comparison `iprf_iff_nprf` connects the eight rules stated here to that kernel. These are attributed existing results; the present validation refinement neither adds a new Lean theorem nor claims that the Python implementation has been formally verified in Lean. Its separate checking argument concerns the representation and validation of certificates for this unchanged mathematics.

3 A smaller validation boundary

The mathematical proof kernel is the function `c`. It is copied without a syntax-tree change from the 897-byte control. The simplification is entirely before that kernel: establish the exact raw data language once, then specialize the public-goal validator to that established domain.

3.1 Recognizing integer/list trees without a recursive type mode

Let $\mathcal{J}$ be the finite values returned by the standard Python JSON decoder. Its possible node types are exact integers, exact lists, strings, dictionaries, Booleans, floating-point numbers, and `None`. The parser's additional `NaN` and infinity constants, when accepted, are floating-point numbers. Let $\mathcal{L} \subseteq \mathcal{J}$ be the least class containing every exact integer and every finite list whose elements lie in $\mathcal{L}$.

Let Σ consist of the ten decimal digits, left and right square brackets, comma, space, and minus sign. Define

$$R(x) \quad \Longleftrightarrow \quad \texttt{repr}(x) \text{ contains only characters in } \Sigma.$$

In Python, `not repr(x).strip('0123456789[], -')` tests precisely this condition: removing allowed characters from both ends leaves an empty string exactly when no forbidden character occurs anywhere.

Lemma 3.1 (Exact raw-domain recognition)**.** *For every finite JSON-decoded value x for which the relevant operations finish, $R(x)$ holds if and only if $x \in \mathcal{L}$.*

Proof. An exact integer has a decimal representation with an optional minus sign. A list representation adds only square brackets, commas, and spaces to the representations of its elements. Structural induction therefore proves the forward construction $x \in \mathcal{L} \Rightarrow R(x)$.

For the converse, every disallowed node contributes a forbidden character which an enclosing list preserves. A string contributes a quote character; a dictionary contributes a brace; `True`, `False`, and `None` contribute letters. A finite floating-point representation contains a decimal point or an exponent marker; infinities and NaNs contain letters. Consequently no tree containing any such node satisfies R. The argument concerns values produced by the standard JSON decoder, not arbitrary Python objects with user-defined representation methods. □

The new guard applies to the *whole decoded input before popping the goal.* Thus an unused prefix cannot hide an invalid value. Top-level integers may pass the raw-domain test, but fail the subsequent list operation; an empty top-level list similarly fails when its goal is popped. Exceptions reject, as in the inherited protocol.

3.2 Specializing the scoped goal validator

The old validator `V(a,d)` has two modes: $d = 0$ recursively checks raw integer/list types, while $d \geq 2$ checks public syntax at opener depth $d - 2$. The new `v(a,d)` has only the latter task and starts directly at opener depth 0.

On the already established domain $\mathcal{L}$, multiplying a node by zero distinguishes the cases: integers yield integer zero and lists yield an empty list. Thus `a * 0 == 0` selects exactly the numerical-leaf case. For an integer leaf, the new scoped condition is

$$d \bmod 2 = 1 \quad \text{and} \quad (a = 0 \text{ or } (2 < a < d \text{ and } a \bmod 2 = 1)).$$

For a list, the permitted arities are 1 and $2 + d \bmod 2$, together with a diagonal triple in the full native profile. Its children are checked at depth $d + 5 - |a|$. This is the inherited arity convention, including its dummy protected slots; it is not a new binding discipline.

Lemma 3.2 (Depth-shift specialization)**.** *For each $a \in \mathcal{L}$ and nonnegative opener depth d, the new* `v(a,d)` *successfully recognizes the same scoped public syntax as the old* `V(a,d+2)`*. The Eq/D restriction satisfies the analogous claim after removing the public diagonal-Q alternative from both validators.*

Proof. Induct on the input tree. Adding two to depth preserves parity; the old bound $a < (d + 2) - 2$ becomes $a < d$. Raw-mode alternatives are false because the old depth is positive. At a list, both validators impose the same arity condition. The old child depth is $(d + 2) + 5 - |a|$, exactly two greater than the new child depth, so the induction hypothesis applies. For an empty list, a short-circuited diagonal access may raise rather than return false. This is rejection in both checker contracts. The statement is about successful recognition, not total Boolean functions on all objects. □

Theorem 3.3 (Preservation of the validated presentation)**.** *Under the finite-certificate mathematical interpretation of the programs, the revised full native checker and its 897-byte control recognize the same valid public proof certificates. The corresponding Eq/D programs also agree. Hence each revised checker presents the same native consequence relation and inherits the same whole-domain bi-interpretation with iPRA.*

Proof. The old raw validator recognizes exactly $\mathcal{L}$. The old split check on the remaining prefix and canonical goal is equivalent to requiring that the original whole input belong to $\mathcal{L}$ and that the goal have the required canonical syntax. Lemma 3.1 establishes the former condition in the new program; Lemma 3.2 establishes the latter. Both then pop the same goal, expose the same packet stream in the same order, and execute the unchanged kernel with the same initial context and freshness bound. All successful kernel steps therefore coincide. No axiom, introduction mode, scope check, or inference capability is added. The inherited soundness and completeness theorem transfers to the revised program. Composing the inherited native/ordinary and ordinary/iPRA interpretations gives the final claim. □

Operational limits and diagnostics. The theorem does not assert identical behavior at every Python resource limit. The old raw traversal uses Python recursion; the new representation test has different implementation limits. An existing valid computation certificate with an unused integer/list prefix nested 600 levels is accepted by the new program but exhausts the old raw validator's recursion limit in the tested environment. Both still enforce the same mathematical data language. Conversely any resource failure rejects, and no finite-hardware program accepts every arbitrarily large finite proof. The error-message text is retained verbatim, but malformed values can reach a different exception class. The acceptance convention remains normal exit with exactly `ACCEPT` followed by a newline.

4 The complete standalone implementation

The source below is the entire trusted checker apart from the Python runtime and its standard JSON module. The selected program imports no producer, axiom database, reference implementation, or test module.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d else d % 2 and
      i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) % 2 == u) and k)], k)

def v(a, d=0):
    return d % 2 and (not a or (2 < a < d and a % 2)) if a * 0 == 0 else (len(a) in (1, 2 + d % 2) or a == [a[0]] *
      3) and all((v(b, d + 5 - len(a)) for b in a))
q = json.load(open(0))
if repr(q).strip('0123456789[], -') or not v((a := q.pop())):
    raise ValueError('The certificate must use integer/list syntax and have a canonical closed goal')
q = q.pop
c(a)
print('ACCEPT')
```

The four packet fields are $[m, s, u, t]$, read from right to left in the outer input list. The checker first verifies the major m. A matching equation major supplies transport; a matching diagonal Q major supplies QE. A proved D major exports its closure under a fresh parameter. A neutral major with truthy u supplies a computation equation; a neutral major with falsy u supplies an introduction. The falsy values [] and 0 distinguish the case and equation-progress choices. Only a literal equation obtains the progress assumption. The exact packet conventions and the inherited certificate decoder are included with the sources.

Certified zero instances and the syntactic exclusion of a literal-zero transport source are inherited implementation devices. Their admissibility is not a consequence of the new representation lemma. The required mathematical arguments are retained in Appendices G and F; the imported source snapshots identify the current matched control. New certificates need no translation to use the revised front end.

4.1 Why a tempting additional reduction was rejected

At entry to the local checker, one has $k = 1 - |g|$, suggesting that k could be removed as a separately transmitted argument. That alone does not eliminate its role. The nested opener closes over the local k, which is decremented *before* the final goal and hypothesis openings. Those calls must admit the newly allocated name.

Replacing that changing bound by one fixed expression gives a smaller program which accepts simple computation certificates but rejects existing contextual-induction and transport-under-binder certificates. A correct version which computes the entry value from g and retains the decrement is eight bytes larger than the original. The regression receipt is included as evidence of a rejected attempt; no incorrect experimental checker is part of the delivered code. The context-length invariant itself was already recorded in the earlier local-scopes package.

5 Measurements and validation

5.1 Measurement convention

Measurements use CPython 3.12.14 and zlib 1.3.2. Source is parsed, docstrings are removed, and `ast.unparse` is used with one final newline. The principal coordinates are UTF-8 bytes, significant Python tokens, and AST nodes. Significant tokens omit encoding, layout, comments, and the end marker. Gzip uses compression level 9 and a fixed zero timestamp. All source files measured here include their input and output interfaces. The original diagnostic sentence has not been shortened to manufacture a byte saving.

Complete standalone program	Bytes	Tokens	AST	Gzip-9
Full Eq/Q/D control	897	401	468	505
Full Eq/Q/D refinement	870	383	442	507
Eq/D control	878	390	454	498
Eq/D refinement	849	370	426	499

For full Eq/Q/D, the reductions are 3.01% in bytes, 4.49% in tokens, and 5.56% in AST nodes. For Eq/D they are 3.30%, 5.13%, and 6.17%. The compressed programs grow by two bytes and one byte, respectively; compression was not the optimization objective. The Eq/D profile changes only the public formula grammar: Q remains an intermediate resource and is definable publicly by D with an unused witness variable.

The optional lexical variant filters the raw input text before JSON parsing, using the digits, brackets, comma, minus sign, and the four JSON whitespace characters. The JSON parser checks the placement of these characters. It has the same intended type boundary and a direct lexical proof, but at 885 normalized bytes it is larger than the selected 870-byte representation variant. It is included for comparison rather than called a second record.

5.2 What the executable checks establish

The reproduced comparison has zero discrepancies. For each public profile, it checks **21,024** JSON-derived values against an independent integer/list predicate and performs **240,120** old/new scoped-validator comparisons. The 101 input fixtures give 21 acceptances and 80 rejections for full Eq/Q/D, and 16 acceptances and 85 rejections for Eq/D, matching their controls. Eleven disallowed unused-prefix types are rejected in ordinary and optimized Python execution. The corpus includes one metadata-shaped JSON object as a deliberate top-level rejection. The proof-kernel AST is unchanged in both profiles.

The proof corpus exercises computations, contextual equation induction, transport beneath D and Q, closure use, compound case introduction, zero-source recovery, and rejection of unproved majors and ill-scoped names. Additional adversarial cases put disallowed JSON values into unused prefixes, where a goal-only check would miss them. The validator comparisons include arrays of incorrect arities, empty nodes, aliases, and bound-slot limits. Runs with `python3 -O` verify that acceptance does not rely on assertions.

Finite tests support the implementation theorem; they do not prove it or constitute a Lean verification. The original incoming package's supplied reproduction was also run successfully before changing anything: 642 process invocations, 18,060 scoped-validator comparisons, and 3,834 raw-type comparisons, all with the expected outcomes. Those are baseline receipts, separate from the new checks reported above. The

current Lean sources were inspected but not rebuilt in this work, and no Lean/Python correspondence theorem is claimed.

6 Further research questions

1. Can the mathematical core be simplified while also reducing a fully validated checker under a fixed certificate contract? A proposed removal must recover the deleted rule and preserve both interpretation composites; shorter surface syntax by itself does not settle the question.
2. Can the arity decoder, public validator, and successful-trace invariant be formalized together, then connected to the existing Lean derivability relation? This would address the main remaining trust gap for the Python implementation without changing iPRA's mathematical presentation.
3. Can numerical terms serve as atomic zero assertions, replacing an equation $s = t$ by a fixed scalar distance term $\delta(s, t) = 0$, at lower *total* trusted cost? The fixed expansion of δ and the internal reflection and round-trip proofs must be counted; adding an uncharged equality-test primitive would merely move complexity.
4. What restricted class of representations and implementations would make a meaningful lower-bound theorem possible? The present collection gives best observed coordinates in particular protocols, not a representation-independent lower bound for axiomatizations.

Artifact guide and attribution

The archive contains the compiled report, its complete expanded LaTeX source, the editable component sources, full and Eq/D standalone checkers, an annotated implementation, a lexical alternative, matched frozen controls, proof examples and adversarial cases, reproducible measurements and checks, and source provenance. Run `python3 reproduce.py` for the executable evidence and `python3 build_report.py` to compile the report with `pdflatex`.

The mathematical construction, certificate kernel, and inherited appendix proofs come from the pinned ProveIt development under its MIT No Attribution license. Their attribution is retained nonetheless. The contribution of this note is the smaller validation boundary and its correctness comparison. The following appendices retain their source terminology, including words such as "new" relative to their original reports; those words do not reclassify the inherited results as new work here. Parallel reviews in the evidence folder are assistant reviews, not external human reviews.

A Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

A.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

A.2 Exact equivalence and the unused-witness construction

Lemma A.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem A.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y} E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition A.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

A.3 Induction and numerical cases separate cleanly

Lemma A.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma A.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

A.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t, u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\,(x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

A.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$
$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)\big). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem A.2 preserves every native sequent.

A.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

B Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section A recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

B.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem B.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

C Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [3] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

C.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

C.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

C.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

C.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

C.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

D From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [4]. In the scalar application used here, every function occurring in the proof

is already a scalar term: predecessor is supplied by Appendix C, the selector is $C(n,a,b) = I(n,a;u.b)$, addition is $x+y = I(y,x;u.Su)$, and truncated subtraction is $d(x,y) = I(y,x;u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

D.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0,a,b) = a, \qquad C(Sx,a,b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n,\vec{p})$:

$$\forall \vec{p}\,\left[(E(0,\vec{p}) \wedge \forall n(E(n,\vec{p}) \longrightarrow E(Sn,\vec{p}))) \longrightarrow \forall n\, E(n,\vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0,\vec{p}), \qquad \Gamma, E(z,\vec{p}) \vdash E(Sz,\vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n,\vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

D.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u,0,1), \qquad K(u) = C(u,1,Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

D.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

D.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

D.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

D.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at

zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

E Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [5]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

E.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v + 1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n + 1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k + 1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a, b)) = a + b$, and therefore

$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

E.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$
$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

E.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section A give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

F Removing literal zero from the source of transport

The following is a restriction of a mathematical inference rule, rather than a change in the spelling of zero. Throughout this section, "nonzero source" means *not syntactically the single term* 0. Every variable and every term headed by S or I is a permitted source, including terms whose numerical value is zero.

Let $\mathcal{C}$ be the native scalar calculus with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

the two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

formulas generated by equality and the Q, D binders, and the Hyp, QE, DE, case-DI and equation-progress-DI rules stated in the main inventory. The same argument applies to the original presentation with D object formulas and Q resources in contexts. Let $\mathcal{C}^-$ replace unrestricted transport by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr}^-, \qquad s \not\equiv 0. \tag{Tr$^-$}$$

Here F is an arbitrary native numerical template, and $s \not\equiv 0$ is a condition on finite term syntax. All other rules are unchanged.

Theorem F.1 (Literal-zero source elimination)**.** *For every native context Γ and formula A,*

$$\Gamma \vdash_{\mathcal{C}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A.$$

In particular, the ordinary whole-domain realization of $\mathcal{C}^-$ is bi-interpretable and equiconsistent with full first-order iPRA through the same interpretations as the native scalar core. No change of numerical domain or interpreted equality is needed.

We prove the theorem without assuming arbitrary numerical substitution admissibility for $\mathcal{C}^-$: such substitution can turn a variable source into the forbidden literal zero. Every occurrence of a helper equation at an arbitrary term is either obtained directly from its displayed derived rule at that term, or from a derivation whose transport sources retain an S or I head under the indicated substitution.

F.1 Structural operations, cases, and equation induction

Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. These operations do not turn a nonzero source into literal zero. As usual, fresh parameters are renamed before a cut. We also use the following restricted substitution fact: a capture-free substitution is admissible for a particular finite derivation if it leaves every primitive transport source in that derivation syntactically different from zero. This fact follows by the same induction; it is not unrestricted substitution admissibility.

Lemma F.2 (Cases and equation induction need no transport)**.** *In $\mathcal{C}^-$ the following rules are derivable:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)} \quad \mathsf{NC}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \quad \mathsf{EI}.$$

In NC, A is any native formula; in EI, E is one literal equation. The parameter z is fresh for Γ, t, and the fixed parameters of the matrix. Other parameters stay fixed throughout EI.

Proof. Choose an unused witness coordinate x. In NC, weaken the successor proof by $A(0)$ and apply case-DI, with witness zero, to obtain $D_{x,y}A(y)$. DE opens $Q_y A(y)$, because x is unused. QE with the given base proof and target t yields $A(t)$, and DE closes. For EI use progress-DI instead, with the given equation step, and use the same DE/QE construction. No transport rule occurs. □

These rules will be applied to unconditional helper equations before they are weakened into the context containing $0 = t$. We never use NC with a case parameter occurring in its retained context.

F.2 The equality operations available immediately

Lemma F.3 (Reflexivity and safe equality operations)**.** *Reflexivity $r = r$ is derivable for every term r. From $p = q$ with $p \not\equiv 0$, one can derive $q = p$ and $H(p) = H(q)$ for any numerical template H. Every direct computation can be used in either direction as an equation; using the reversed equation as a transport major is permitted only when its new source is syntactically nonzero.*

Proof. Put $b = I(0, r; u.0)$. The computation $b = r$, used as both major and minor of transport with template $v = r$, gives $r = r$. Its source is the nonempty iterator term b. For symmetry of $p = q$, transport the reflexivity $p = p$ through the template $v = p$. For numerical congruence, transport the reflexivity $H(p) = H(p)$ through $H(p) = H(v)$. Both use the permitted source p. Every computation equation has an I-headed left side, so its symmetry follows by the same construction. □

An occurrence-selective template may keep some occurrences of a term fixed while abstracting other occurrences. This is ordinary capture-free template substitution, not an extra inference rule.

Lemma F.4 (Joining two nonzero terms known to be zero)**.** *Suppose $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$. Then $\Gamma \vdash p = q$ using only Tr^- and computation.*

Proof. Let

$$z_0 = I(0, 0; u.0), \qquad b_p = I(0, p; u.0).$$

Computation gives $b_p = p$. Start with $b_p = b_p$ and use the major $p = 0$ in the template $I(0, v; u.0) = b_p$. This gives $z_0 = b_p$. Symmetry at the nonzero source z_0 gives $b_p = z_0$. Now transport through $v = z_0$ using the computation $b_p = p$, whose source is b_p, to obtain $p = z_0$. The identical construction gives $q = z_0$. Symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ gives $p = q$. The sources used are p, q, z_0, b_p, b_q; all are syntactically nonzero. No reverse transport from zero has been used. □

The zero-join lemma is deliberately placed before the next construction. The positive case of its singleton test needs precisely this lemma; ordinary transitivity through zero is not yet available.

F.3 A singleton-one test without predecessor

Define the finite scalar terms

$$\begin{aligned} C(v, a, b) &:= I(v, a; u.b), & Z(v) &:= C(v, 1, 0), \\ K(v) &:= C(v, 0, 1), & H(v) &:= C(v, 1, 2), \\ N(n) &:= I(n, 0; u.Z(u)), & R(n) &:= I(n, 0; u.H(u)), \\ L(n) &:= N(R(n)). \end{aligned} \tag{Tests}$$

The update variable in C is unused. Here $1 = S0$ and $2 = S1$. The standard-model description of R is $0, 1, 2, 2, \ldots$; the proof below establishes the needed equations internally and does not assume this description.

Lemma F.5 (Uniform test equations)**.** *The following equations have derivations in $\mathcal{C}^-$, uniformly in the displayed argument:*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$

$$N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad \boxed{L(Sv) = Z(v)}. \tag{Singleton}$$

All transport sources in these helper derivations have an I or S head, except for the explicit nonzero endpoints in the zero-join lemma; those endpoints also have an I head in this application.

Proof. For $H(v) = SK(v)$ use NC. At zero both sides reduce to 1; at a successor both reduce to 2. To join two equations $p = j$, $q = j$ with $j = 1$ or $j = 2$, reverse $q = j$ at its nonzero source q, then transport at the nonzero numeral j. Each reduction of these particular terms is a forward replacement of an iterator computation. This proves the helper at an arbitrary target v by NC, without substituting zero into a variable-source transport.

The equations $N(1) = 1$ and $N(2) = 0$ are finite computations: $N(0) = 0$, $N(1) = Z(N(0)) = 1$, and $N(2) = Z(N(1)) = 0$. Every replacement source in these calculations is an iterator term. Likewise $R(1) = H(R(0)) = H(0) = 1$.

Computation and the first helper give, for any v,

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Use this equation at the nonzero source $R(Sv)$ in $R(SSv) = H(R(Sv))$. The resulting right side is $H(SK(R(v))) = 2$ by the successor computation of C. Thus $R(SSv) = 2$. These substitutions are all at I-headed sources. Substitute $R(1) = 1$ and $R(SSv) = 2$ inside N and use the two computed values of N to obtain $L(1) = 1$ and $L(SSv) = 0$.

Finally apply NC to the matrix $L(Sv) = Z(v)$. At zero, $L(1) = 1$ and $Z(0) = 1$ are joined through the nonzero numeral 1. At a successor, $L(SSv) = 0$ and $Z(Sv) = 0$ are joined by Lemma F.4; both left sides are nonempty iterator terms. This proves the boxed equation at every target. Its use is therefore an instance of a displayed derived rule, not an appeal to unrestricted substitution admissibility. □

Lemma F.6 (An unconditional zero-filter identity)**.** *For every term t,*

$$t = C(Z(t), t, 0) \tag{Filter}$$

is derivable in the empty context of $\mathcal{C}^-$.

Proof. Apply NC to $v = C(Z(v), v, 0)$, with the target t. At zero, forward computations give $C(Z(0), 0, 0) = C(1, 0, 0) = 0$; reverse this equation at its nonzero left side. At Sv, forward computations give $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$; again reverse at the nonzero iterator left side. Thus the base and successor proofs have no retained assumption depending on the case variable. All numerical arguments of the final NC target are permitted. In particular, this identity is proved before any assumption $0 = t$ is used. □

F.4 Turning a zero-source equality around

Lemma F.7 (Zero-source symmetry)**.** *For every term t,*

$$0 = t \ \vdash_{\mathcal{C}^-} \ t = 0. \tag{ZS}$$

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Work in the fixed context $\Gamma = \{0 = t\}$ and put

$$A(n) := I(n, t; u.Su).$$

We first prove $n = A(n)$ by EI, keeping t fixed.

For the base, computation gives $A(0) = t$. Its symmetry gives $t = A(0)$. Since $t \not\equiv 0$, transport at source t in the retained assumption $0 = t$ gives $0 = A(0)$.

For the step, retain $z = A(z)$ with z fresh. This source is a variable, hence is permitted. Numerical successor congruence from Lemma F.3 gives $Sz = SA(z)$. Computation gives $A(Sz) = SA(z)$, and its symmetry gives $SA(z) = A(Sz)$. Its source $SA(z)$ is also permitted, so transport in $Sz = SA(z)$ gives

$Sz = A(Sz)$. This is exactly the EI step. The induction hypothesis is used at the same fixed parameter t.

At target 1, EI gives $1 = A(1)$. Forward computation gives $A(1) = St$: the successor equation produces $SA(0)$, and the nonzero-source computation $A(0) = t$ is used inside S. Transport at source $A(1)$ consequently gives

$$1 = St.$$

Its source 1 is permitted. Numerical congruence for L gives $L(1) = L(St)$. Use $L(1) = 1$ at source $L(1)$, and $L(St) = Z(t)$ at source $L(St)$, obtaining $1 = Z(t)$. Symmetry at source 1 gives $Z(t) = 1$.

Weaken the unconditional Filter identity into Γ. Transport at the iterator source $Z(t)$ changes its right side to $C(1, t, 0)$. The latter computes to zero, and its source is again an iterator term. The resulting equation is $t = 0$, as required. □

This proof uses cases only for unconditional helper equations. The sole induction with the retained assumption $0 = t$ has the literal equation $n = A(n)$ as invariant. It does not perform a forbidden case distinction while silently changing the retained context.

F.5 Recovering equality algebra and formula transport

Lemma F.8 (Full numerical equality algebra)**.** *The calculus $\mathcal{C}^-$ admits unrestricted symmetry, numerical congruence, and transitivity. In particular it admits root replacement*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}.$$

Proof. Symmetry uses Lemma F.3 when the source is not zero and Lemma F.7 otherwise.

For numerical congruence from $s = t$, a nonzero s is handled by Lemma F.3. If $s \equiv t \equiv 0$, use reflexivity. If $s \equiv 0$ and $t \not\equiv 0$, first derive $t = 0$, apply permitted numerical congruence at source t to obtain $H(t) = H(0)$, and use the now established full symmetry.

For transitivity from $r = s$ and $s = t$, a nonzero middle term s is handled by one permitted transport. If $s \equiv 0$ and either endpoint is literally zero, a given premise is already the conclusion. Otherwise $r, t \not\equiv 0$; reverse $0 = t$ to get $t = 0$ and apply the zero-join lemma to $r = 0$ and $t = 0$. Root replacement follows by symmetry and transitivity. These constructions are acyclic: zero-source symmetry was proved before any unrestricted numerical congruence or transitivity. □

Lemma F.9 (Full transport)**.** *For every native numerical template $F(v)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in $\mathcal{C}^-$, without a source restriction.

Proof. Use external induction on the finite number of formula binders in F, uniformly in retained contexts and fixed numerical parameters. For an equation $F(v) \equiv A(v) = B(v)$, use unrestricted numerical congruence to obtain $A(s) = A(t)$ and $B(s) = B(t)$. Join these with $A(s) = B(s)$ using the established equality algebra.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh a, retaining $Q_y B(s, a, y)$. Introduce the desired $F(t)$ by case-DI with witness a. Its local assumption is $B(t, a, 0)$. Full symmetry gives $t = s$, and the external induction hypothesis for the smaller matrix transports the assumption to $B(s, a, 0)$. QE gives

$B(s, a, Sz)$, and the induction hypothesis in the forward direction gives $B(t, a, Sz)$. Close case-DI and DE. All new parameters can be chosen fresh for the source equality and the retained context.

If Q is first-class formula syntax, first observe that its case introduction is derived without transport: introduce $D_{u,y}B(y)$ with unused u by case-DI and eliminate it with the one-node continuation $Q_yB(y)$. For $F(v) = Q_yB(v, y)$, use this derived introduction for the target. Convert its base assumption backwards by the induction hypothesis, apply the old Q resource, and convert the successor conclusion forwards. This is the same smaller-matrix argument and uses no new numerical induction. □

Proof of Theorem F.1. Every restricted derivation is an unrestricted one. Conversely, replace each unrestricted transport node by the derivation supplied by Lemma F.9, recursively translating its premises first. Other inference nodes are unchanged. The result has the same context and conclusion. Thus the full native consequence relation, not just closed theorems, is preserved. The existing native/ordinary and ordinary/iPRA interpretations and both identity composites therefore transfer unchanged. □

F.6 Compatibility with restricted computation certificates

Some compact checkers require the initializer of a computation packet to have nonempty syntax, and encode a successor counter by a nonempty predecessor. These are certificate restrictions, not restrictions on the two mathematical computation schemas. They do not introduce a circular dependence on zero-source transport in the preceding proof.

Use the inherited certified-zero term

$$c_0 = I(2, 1; u.0).$$

Its equation $c_0 = 0$ is a directly admitted successor computation: the predecessor and initializer are both 1. For an arbitrary computation instance with literal-zero initializer or literal-zero successor predecessor, form its whole equation as a template $J(v)$, replacing just these offending occurrences by v. The instance $J(c_0)$ is a directly admitted computation. Transport through this equation template using $c_0 = 0$ yields $J(0)$, the desired original instance. Both computation premises are admitted directly, and the only transport source is the nonempty term c_0.

This is exactly the inherited three-node normalization in `Research/certified_zero/tests/producer.py`, function `expand_computation`: one transport and two computation premises. Its source is c_0, even when both defects are corrected simultaneously. Passive zero occurrences in an update are left alone. Thus all computation instances used in the mathematical proof can be expanded before compiling to the source-restricted checker.

F.7 Scope of the result

The new statement is the admissibility of all omitted literal-zero source transports. The native term/formula language, arithmetic schemas, and target theory remain the inherited ones. This theorem justifies replacing a two-way equality/Q major-pattern test by a source-discriminated test in a suitable certificate protocol. It does not assert a minimum proof-checker size, a shortest proof expansion, or a new characterization of the primitive recursive functions. The displayed proof is a written universal derivation; executable certificates, when supplied, validate particular instances rather than substitute for this argument.

G A uniform arity encoding and its checking theorem

The arithmetic and object logic remain the scalar-iteration Eq/D calculus specified above. The change is in its finite certificate representation. A paired formula is unary, an equation is binary, and an internal closure is ternary:

$$\mathsf{D}_{x,y}B \longleftrightarrow [B], \qquad s = t \longleftrightarrow [s,t], \qquad \mathsf{Q}_y B \longleftrightarrow [B,B,B]. \tag{Arity encoding}$$

The three copies in a closure are representation redundancy: only the third has mathematical content. Their benefit is that tripling a unary major constructs exactly its closure. Tripling any other nonempty formula array produces an array of arity at least six, which will denote truth. This makes the other major cases neutral without a separate test.

The previous generated-assumption and constructed-computation interfaces are inherited from the corpus [6, 7]. The new encoding makes a single arithmetic expression, $5 - \text{arity}$, perform the necessary depth adjustments. Its soundness depends on an explicit interpretation of surplus syntax. It would not be sound to drop arity checks while continuing to treat every surplus array as an unconstrained new connective.

G.1 The complete sorted interpretation of arrays

Write $\top$ for the fixed equation $0 = 0$. A protected numerical slot has width two. The numerical root has depth three: the coordinates $0, 1, 2$ all denote zero, and under p protected slots the depth is $3 + 2p$. Coordinates $3 + 2j$ and $4 + 2j$ name slot j, counted from the innermost slot, for $0 \le j < p$. Formula depth is $2p$ under the same p slots. Negative coordinates admitted by the current global floor name rigid numerical parameters.

The decoder is sorted by the position in which a tree is used. Let ρ be its ordered environment of protected numerical slots. The notation (a, ρ) means prepend a slot of value a. A superscript $\sharp$ denotes a child decoded in the indicated environment.

Numerical arity	Meaning	Relevant child environment
0	0	No child
1: $[b]$	$Sb^\sharp$	$(0, 0, \rho)$
2	0	Annotations ignored
3: $[n, a, h]$	$I(n^\sharp, a^\sharp; v.h^\sharp)$	$(0, \rho)$ for n, a; (v, ρ) for h
≥ 4	0	All children ignored

Formula arity	Meaning	Relevant child environment
0	$\top$	No child
1: $[B]$	$\mathsf{D}_{x,y}B^\sharp$	(y, x, ρ)
2: $[s, t]$	$s^\sharp = t^\sharp$	ρ numerically
3: $[P, U, B]$	$\mathsf{Q}_y B^\sharp$	(y, ρ) for B
≥ 4	$\top$	All children ignored

The first two entries of a ternary formula are annotations. A canonical closure copies its body into both of them, as in (Arity encoding). Successfully admitted scalar formula padding also denotes $\top$. An internal closure has the ordinary meaning $B(0) \to \forall y B(y)$; it is not an equation and is not eligible for compound induction.

Canonical encoded object expressions use arrays and integer coordinates, respect these slots, and contain no high-arity node. The checker itself still generates high-arity truth entries in administrative contexts, even during a canonical proof. In particular, constructing a successor beneath positive bound coordinates requires passing through two dummy zero slots. Constructing an iterator requires one new slot: zero in its counter and seed, and its genuine state variable in the update. Rigid negative parameter names are unaffected by these shifts.

The additional raw shapes are certificate aliases with the displayed meanings. They are not independent predicate or function symbols added to the object theory. Our exact completeness statement concerns the canonical native language. We do not require every possible raw spelling of a semantic alias to be interchangeable under the same raw certificate.

G.2 One depth calculation replaces the lookup table

For every array the opener recursively adds $5 - \text{arity}$ to its depth. The meaningful cases are

arity	1	2	3
depth increment	4	3	2.

Unary D adds two numerical binders; unary successor adds two dummy zero slots. Binary equality changes formula depth $2p$ to numerical depth $3 + 2p$. Ternary Q adds one numerical binder; ternary iteration adds one state slot, or one dummy slot in its counter and initializer. The same calculation therefore covers all relevant constructors.

Binary numerical annotations denote zero, so their parity-changing traversal has no mathematical effect. Empty arrays have no children. At arity at least four, the entire node already denotes a constant. Even a zero or negative depth increment below such a node cannot expose one of its ignored children as a meaningful variable. Traversal may still reject an alias; it cannot change its stipulated interpretation.

This is a semantic assertion, not raw idempotence of the opener. For example, at floor -1, opening a six-element array of zeros with image -1 can yield six copies of -1, while a later no-image opening of that result fails at a negative child depth. Both formula arrays denote $\top$. The proof below only concerns successful calls at meaningful sorted positions. Canonical input object expressions contain no surplus arrays; the administrative exports that generate them are justified separately as truth entries.

G.3 Opening and scope

Write $i(a, t, d)$ for the executable opener at floor k. It preserves a scalar when $k \leq a < d$. At a numerical position, every other relevant scalar is the distinguished substitution hole; the supplied image is checked at numerical root depth three before insertion. At a formula position scalar padding is truth and carries no numerical hole. Missing images at used numerical holes reject.

Lemma G.1 (Semantic opening)**.** *For every successful call at a meaningful formula root of depth $2p$ or numerical root of depth $3 + 2p$, with $p \geq 0$, decoding its result is capture-free filling of the distinguished hole in the decoded input template. These depths protect p numerical slots; admitted rigid parameters are preserved. If an image is semantically unused, zero may be chosen as its mathematical representative. Recursive calls at negative depths inside ignored annotations need no independent sorted interpretation.*

Proof. Induct on the finite raw tree. At a relevant scalar, the interval comparison is exactly the protected-coordinate test. At a meaningful unary, binary, or ternary node, the preceding tables supply precisely the required additional slots. At numerical binary annotations, high-arity nodes, and ignored

closure fields, the relevant mathematical meaning is constant or absent, so the conclusion is immediate regardless of traversal inside the annotation.

An inserted image is checked at its own numerical root. Its free parameters are rigid negative names, and it has no unbound positive coordinate there. Insertion beneath protected slots therefore captures none of its parameters; its internal binders retain their relative indices. This proves the inductive step for substitution as well as for mere traversal. □

A successful no-image check consequently preserves the interpretation of its checked closed expression. Raw structural equality of checked outputs implies equality of their decoded meanings. The checker does not infer equality merely from a semantic comparison of two different programs or numerical terms.

The global floor decreases by one at each non-hypothesis continuation. If its old value is K, the integer $K-1$ is the designated fresh eigenparameter. With integer coordinates it is the only new name. If fractional coordinates are accepted, other names in $[K-1, K)$ may also occur in the finite certificate. These are additional fresh parameters. They must not be silently equated to the designated integer eigenparameter.

Previously checked relevant goals and retained hypotheses mention none of these new names: their earlier no-image checks would reject a relevant numerical occurrence below the old floor. Coordinates inside ignored annotations do not contribute free variables to the decoded formula. This is the freshness invariant used in every rule below.

G.4 A computation hypothesis only needs a zero instance

The shared continuation can open a generated arithmetic equation at a fresh parameter. That equation need not be a theorem uniformly in the new parameter. The following admissible rule is sufficient.

Lemma G.2 (Supply with a known zero instance)**.** *Suppose $\vec{a}$ is fresh for Γ, F, the calculus proves $H(\vec{0})$, and $\Gamma, H(\vec{a}) \vdash F$. Then $\Gamma \vdash F$.*

Proof. Substitute zero for every member of $\vec{a}$ in the second derivation. Freshness leaves Γ, F unchanged. Cut in the derivation of $H(\vec{0})$. Both substitution and cut are admissible in the native calculus. This is a finite proof transformation and does not use a consistency assumption. □

All names newly admitted when the floor changes belong to $\vec{a}$; for noninteger aliases this includes the finite collection in $[K-1, K)$ that actually occurs. On decoding, their simultaneous zero substitution agrees with opening the generated expression at zero using the old floor. Previously admitted parameters are preserved. New rigid names and unprotected placeholders both become zero. The opener may keep different raw zero spellings, but their decoded values agree.

The constructed-computation packet supplies a triple $s = [n, b, r]$ and a mode u. In successor mode its generated equation has raw form

$$\big[\,[[n], b, r],\ i(r, s, 3)\,\big], \tag{Step supply}$$

while in zero mode it has form

$$\big[\,[u, b, r],\ b\,\big]. \tag{Zero supply}$$

Here successor mode means that u is truthy; the zero mode has falsy u. These expressions describe actual arrays, before their final opening. In (Step supply) the counter is constructed as a successor, rather than accepted on an unsupported assertion that it is one. This construction is inherited from the corresponding draft interface [7].

Lemma G.3 (Sound constructed computation)**.** *Every successful arithmetic supply has a decoded zero instance derivable from the two scalar-iteration computation schemas.*

Proof. First apply the simultaneous zero substitution described above. In zero mode every successful falsy numerical counter denotes zero, so (Zero supply) is an $I0$ instance. In the exact-list profile the relevant accepted aliases are an empty array, numeric zero, false, and numerically equal zero values. Null, strings, and dictionaries cannot enter a meaningful numerical position through an unchecked coercion.

In successor mode the left counter is an actual successor. If the update uses its state in a semantically relevant position, opening it invokes the no-image check of the recursive source s at numerical root three. This checks the entire source, including its update. Only coordinates three and four are aliases for its single state slot. A genuinely escaping coordinate in that update is revisited during the recursive-image check and rejects. Its counter and initializer are likewise closed with their prescribed dummy zero slot.

Inserting the extra dummy slots of the displayed successor does not alter such a checked predecessor: extra unbound slots have value zero, and actual free parameters are rigid. Lemma G.1 therefore identifies the right side of (Step supply) with the update applied to exactly the recursive predecessor value occurring on its left. This is the IS schema.

If the update has no semantically relevant state occurrence, its decoded value is constant in that variable. Its zero-normalized value is the same in the source update and on the right, independently of the unused recursive image; this is again an IS instance. Traversal inside an ignored annotation may still inspect the image and reject. That removes a certificate and does not invalidate the argument for a successful one.

Finally, successful three-field source extraction cannot hide another JSON operator. A scalar is not iterable. A three-character string, or a three-key JSON dictionary, necessarily puts a nonempty string in a relevant source field, which the inspected opener rejects. The successful source is therefore the stated array triple. □

The recursive-image check in the state-dependent case is essential. Merely opening a final displayed equation at zero would not establish that its two state interpretations agree. Combining Lemma G.3 with Lemma G.2 justifies both actual image choices of the common continuation, including the branch that retains a fresh-parameter hypothesis.

G.5 Audit of the logical continuations

Hypothesis closure. The initial opener check and membership comparison select the same decoded context formula. Additional aliases can make a spelling less convenient, but cannot identify an equation with a false denotation solely by semantic evaluation.

Equality transport. The major must match $[s, u]$, and opening the proposed formula template at u must match the target. The continuation checks its opening at s. Lemma G.1 gives ordinary transport along the decoded equality. The argument applies to compound formulas, internal clauses, and truth aliases as well as equations.

Closure specialization. The closure pattern is the triplicate $[t, t, t]$. The guard forces the old instance to have numerical image zero; the new instance may have an arbitrary numerical image. This is exactly $B(0) \to \forall y B(y)$. The zero guard is indispensable. Without it, a closure with false zero instance and true successor instances would turn a proved successor instance into falsity.

Paired elimination. The export is always three concatenated copies of the major. A unary major $[B]$ therefore exports $[B, B, B]$. All three copies are opened identically; the third copy has precisely the witness instance required by paired elimination. The newly allocated witness is fresh for the retained context and conclusion.

An equation major instead exports arity six; a closure major exports arity nine; a larger array exports still larger arity. These are all truth aliases. Thus failed elimination endpoint claims have only the effect of adjoining a theorem. A numeric scalar major, if accepted as formula padding, exports scalar padding after multiplication and also denotes truth. Unsupported types reject.

A ternary formula with unequal annotations still denotes the closure of its third child. It fails the triplicate specialization pattern unless the three fields match. This loses a possible raw spelling, not a mathematical soundness condition. Canonical compilation uses triplicate closures throughout.

Paired introduction. The call `i(*a,t,2)` uses the function's argument count to enforce the relevant outer arity. For a unary goal $a = [B]$, it is exactly `i(B,t,2)`: the universal slot is protected while the existential witness is opened. For arity at least two, there are too many arguments and the call rejects. An empty goal can instead make the legal two-argument call `i(t,2)`. Its original target was already $\top$, so accepting it cannot add a false theorem.

For the unary goal, the subsequent successor opening gives the intended premise $B(t, Sz)$. The local hypothesis is $B(t, 0)$ in case mode and $B(t, z)$ in progress mode. The freshness invariant supplies the usual eigenvariable conditions.

Induction remains restricted. Among meaningful nonconstant matrix constructors, only binary equality has even arity. Unary paired formulas and ternary closures have odd arity and receive case introduction. Empty or surplus arrays that reach the progress branch denote the fixed equation $0 = 0$, whose induction is valid. Scalar padding that cannot provide a length rejects. The broader certificate alphabet therefore adds no induction on a quantified matrix.

Additional fresh parameters. An accepted subderivation can contain newly admitted fractional names, but the older context and target do not depend on them. Replace these auxiliary names by zero when closing the rule, retaining its designated integer eigenvariable where that rule needs one. The common opening of an already checked retained goal leaves its decoded meaning unchanged. This handles the actual global-floor convention without pretending that every admissible negative number is an integer.

G.6 Canonical completeness, including zero witnesses

Rename a finite native proof's eigenvariables to fresh negative integers. Encode its terms with the slot conventions of Section G.1; encode paired formulas as unary and closures as triplicate. Hypothesis leaves consume no instruction. Transport, closure specialization, and paired elimination have their displayed packets. Arithmetic computation leaves use (Zero supply) or (Step supply) and then close the target by hypothesis lookup.

One representational issue requires an actual proof transformation. The packet's truthy witness field selects introduction. A literal zero witness therefore cannot simply be inserted into that field. Define

$$J(t) = I(0, t; v.0).$$

The computation axiom proves $J(t) = t$, and $J(t)$ has a nonempty array representation even when its value is zero. To introduce with a falsy witness spelling t, introduce instead with $J(t)$: transport its

retained assumption to the assumption using t, use the original premise derivation, and transport the resulting successor instance back to the instance using $J(t)$. Derived symmetry supplies the reverse transport. This is a finite transformation of the premise, and the paired conclusion itself is unchanged. If the existential coordinate is unused, any closed truthy witness works without a conversion.

The scalar coordinate 1 is also a semantic zero at numerical root three, but that observation alone is not a completeness proof. Closure specialization and case introduction insert literal raw zero, and distinct zero aliases need not be interchangeable under a fixed raw proof spelling. The computation-backed box gives the required derivation instead of assuming representational interchangeability.

The mode zero selects progress at an equation. A different falsy mode, such as the empty array, selects case. These are certificate control tags, not numerical assertions about the witness. All steps in the compilation are finite and effective.

Theorem G.4 (Conservativity and completeness of the arity checker)**.** *Every accepted certificate has its decoded conclusion provable in the ordinary realization of the native calculus. For every canonical native end sequent, the internal checking judgment and the native calculus have the same consequence relation, in the idealized unbounded integer-and-array implementation. In particular, the public empty-context driver recognizes exactly the canonical closed theorems. The translation in each direction is effective on finite derivations.*

Proof. For soundness, induct on the finite accepting call tree. Hypothesis closure uses Lemma G.1; the logical cases are the audited continuations above. Arithmetic supply uses Lemmas G.3 and G.2. Every discarded annotation is accounted for by its zero or truth interpretation, and every fresh parameter by the scope invariant. For completeness, compile a finite native derivation by the preceding construction, including the boxed-witness transformation. Its canonical input expressions require no surplus-arity aliases or fractional names; generated administrative surplus arrays have exactly the truth meaning already proved. This proves both implications without using a finite test suite as a substitute for induction on derivations. □

The unchanged native calculus is bi-interpretable with full first-order iPRA by the inherited arithmetic and logical bridge [6, 8]. In particular,

$$(\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x,0) \to \forall y B^\circ(x,y)).$$

The whole numerical domain and actual equality are retained. Its internally proved numerical round trips and its formula-composite derivations therefore transfer through Theorem G.4. We have changed the checked representation, not replaced bi-interpretability by standard-model agreement or supplied a new proof of the scalar predecessor theorem.

G.7 JSON and finite-runtime qualifications

The primary profile uses an ordinary list-type test. The smaller byte profile instead tests multiplication by zero against an empty list. Among standard JSON runtime values the latter recognizes exactly lists: numeric values and booleans produce numeric zero, strings produce an empty string, and null and dictionaries raise rather than becoming arrays. This argument assumes ordinary decoded JSON values, not arbitrary Python objects with overloaded methods.

Finite nonnegative real coordinates have the same interval interpretation: $[0,3)$ is zero, and $[3+2j, 5+2j)$ names slot j. Finite negative coordinates in the admitted range are additional rigid names. Equal Python numerical values have the same decoded role, including Boolean aliases and signed zero. Nonfinite values admitted by Python's permissive JSON reader cannot survive a relevant closed numerical check.

They may be filled as template holes or ignored inside a constant annotation; neither makes them unchecked parameters.

The alternative opener based on generic iteration has a separate runtime qualification. Empty strings and empty dictionaries traverse as empty arrays; a dictionary whose sole key is empty traverses as a unary array with an empty child. Dictionary values are not visited. Nonempty strings can rely on CPython's recursion limit to reject, as also occurs in a current corpus comparison profile [9]. The idealized structural-termination claim for the exact-list profile must not be transferred to that generic profile unchanged.

All executable profiles have ordinary finite memory, stack, and input limits. These can reject a sufficiently large otherwise valid proof. The formal consequence relation is the unbounded finite-certificate calculus; the delivered program checks certificates within its runtime resources. The measurements charge the entire delivered checker, while the semantic proof and canonical producer are supplied separately.

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, commit `42a48d2f04ad72036e8d7e365ea91ea08c76a9e3`. https://github.com/VladimirReshetnikov/ProveIt/tree/42a48d2f04ad72036e8d7e365ea91ea08c76a9e3/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt incoming archive, `docs/incoming/ipra_validated_core_2026-11-11.zip`, at the commit above. The exact full native control is copied as `reference/native_typed_full.py`.

[3] ProveIt inherited scalar-predecessor proof, `Research/certified_zero/reference/scalar_predecessor.tex`, reproduced in Appendix C.

[4] ProveIt inherited equality-reflection proof, `Research/certified_zero/reference/equality_reflection.tex`, reproduced in Appendix D.

[5] ProveIt inherited primitive-recursion compiler, `Research/certified_zero/reference/primitive_recursion.tex`, reproduced in Appendix E.

[6] ProveIt inherited generated-assumption and arity certificate development, preserved in `Research/certified_zero/reference/inherited_arity_checker.tex`.

[7] ProveIt research package `Research/constructed_computation`; constructed computation sources are restated in Appendix G.

[8] ProveIt research collection and its package-specific claim records, all at the pinned commit in reference [1].

[9] ProveIt research package `Research/transient_seed`, at the same pinned commit.